COGS 160 — Week 2 AI-Directed Programming

Exercise D: Evidence Search Filter

Student: Dhruv Sood **Date:** 2026-05-03 **Track / Team:** Track 1 — Evidence Explorer (Exercise D)
Files: `filter.html` (implementation), `screenshots/` (visual verification), this submission doc

---

1. Exercise Chosen

Exercise D — Evidence Search Filter. Build a search-and-filter bar for `ka_evidence.html` so a user can narrow ~1,900 K-ATLAS evidence claims by warrant class, credence, topic, status, qualifier, defeat type, study type, support/attack counts, and keyword. Primary user stories from the assignment:

- Architect: *"What does the evidence say about biophilic design and stress reduction?"*
- Philosophy student: *"All contested claims with undercutting defeaters."*

---

2. Context Initialization

Pasted as the first AI message: `context_exD_search_filter.md` (the course-provided context file, full text). Two real data files were attached: `evidence.json` (1,900 records, 5.7 MB) and `argumentation.json` (4.7 MB, not strictly needed for D but available for cross-reference).

The very first thing the AI did was probe the real data instead of trusting the context file. That mattered (see §3 below).

---

3. AI Explanation of Task

> The component is a client-side filter bar that sits at the top of `ka_evidence.html` and reduces the visible set of ~1,900 evidence cards to those matching the user's combined criteria. It runs entirely in memory; the data is already fetched once.
>
> **Key data structures:** A flat array of evidence records. Each record is filterable on ~10 fields. Filtering is conjunctive (AND across fields). Within a field with a free-text input (`finding` , `topic`), match is substring/contains, case-insensitive.
>
> **Fields that matter most:** `warrant_class` , `credence` , `warrant_status` , `qualifier` , `defeat_type` , `support_count` , `attack_count` , free-text `finding` , and **topic** (via `primary_topic` / `topic_labels` , *not* `construct` — see below).
>
> **Hardest part:** Not the filtering logic — that's trivial. The hard part is *which filters are visible by default*. Ten dropdowns is a usability failure; hiding the philosopher's filters is an epistemic failure. Progressive disclosure has to be defended, not assumed.

> **What could go wrong:** (a) Hiding the count of *filtered-out* results, which makes the user think they're seeing the whole evidence base. (b) Free-text on `finding` matching too aggressively (e.g., "stress" matches "distressed" — acceptable, but should be transparent). (c) Combining filters that produce zero results without an obvious "widen this filter" affordance.
>
> **Assumptions to verify before building:**
>
> 1. The `construct` field actually contains ~200 distinct values.
> 2. The enums (`qualifier` , `warrant_status` , `studyType` , `signal`) match the context file.
> 3. There's a viable path for the architect's "biophilia" query.
>
> Probing the real data falsified assumptions 1 and 2:
>
> | Field | Context file claim | Reality in `evidence.json` |
> |---|---|---|
> | `construct` | ~200 distinct values | **All 1,900 are** `"Unknown"` — useless as a filter |
> | `qualifier` | 3 values | 5 (`stable` , `underspecified` , `defeated` , `supported_but_contested` , `review_needed`) |
> | `warrant_status` | 2 values | 3 (`WARRANTED` , `UNGROUNDED` , `DEFEATED`) |
> | `studyType` | 4 values | 28 distinct values (long tail) |
> | `signal` | 4 values | 6 |
>
> So the architect's biophilia query has to route through `primary_topic` / `topic_labels` (e.g., `"Nature & Biophilia → Neural Activity"`), not `construct` . This single discovery rewrites the spec.

---

4. Panel Transcript

Prompt to the panel (copied verbatim from the assignment, then extended with one extra voice — see "Panel notes" below):

> You are running a design panel for a search and filter bar on the Evidence Explorer page. The panelists are: - Jakob Nielsen (usability): What are the most common filtering mistakes? Which filters visible by default vs. "advanced"? - A practitioner (architect) trying to answer: "What does the evidence say about biophilic design and stress reduction?" - A philosophy student trying to find: "All contested claims with undercutting defeaters." - A research librarian (added voice): What query patterns will users actually attempt, and where will they get stuck?
>
> Each panelist: which 3 filters matter most? What filter would be misleading or useless?

Panel notes. I added a fourth voice (research librarian) because the three assignment voices argue from *roles* — usability, practitioner, philosopher — but none of them argue from *observed user behavior*. The librarian is the person who has watched real users fail at exactly this kind of interface, and adding them produced a different axis of disagreement (Boolean operators / saved searches) that the original three didn't surface. Marked as a deviation from the canonical prompt.

Round 1 — Filter rankings

Nielsen (usability): Three rules. One: never put more than four filters above the fold; the others go behind a disclosure. Two: *always* show the filtered count next to the total — "Showing 154 of 1,900" — otherwise people don't know what they're not seeing. Three: text search is the default mental model, so a single keyword box should be the leftmost field. My top three: keyword, warrant_class, credence range. The misleading filter is `signal` — users don't know what "Discussion Level Interpretation" means and they'll filter it out without understanding they're throwing away half the corpus.

Architect: I disagree on `signal`. I want it. When I'm specifying a building, "Direct Measured Result" is the only thing I trust; everything else is a literature claim and I'm liable if I cite it as fact. But I don't need warrant_class — I don't know what "empirical_association" means and I never will. Give me topic, credence, and study type. That's it. And `warrant_status` should be a default checkbox like "hide ungrounded" rather than a dropdown.

Philosophy student: You're both wrong about what matters. The whole point of K-ATLAS is the *contested* claims. I want `qualifier=supported_but_contested`, `defeat_type` (rebutting vs undercutting is the entire research question), and `attack_count > 0`. If you bury those behind "Advanced," you're telling students that disagreement is an edge case. It's not. It's the data.

Research librarian: All three of you are arguing about which fields belong on the page. I want to argue about something none of you mentioned: Boolean operators between filter values. Watch a user type "biophilia OR nature" into your keyword box. Right now that returns zero hits because the substring `"biophilia or nature"` doesn't appear in any finding. Users *expect* OR semantics in keyword boxes — if you don't support them, you're inheriting a 30-year debt of user training and frustrating it. Top three: keyword (with an honest "matches substring, no Boolean" hint underneath), `warrant_class`, and a saved-search affordance so the philosophy student doesn't have to re-construct her contested+undercutting query every visit.

Round 2 — Disagreements (each panelist rebuts another's choice)

Nielsen → Philosophy student: "Contested" is statistically an edge case. Look at the distribution — 1,820 of 1,900 records have empty `defeat_type`. If the median user's median query never touches that field, putting it default-visible is bad design.

Philosophy student → Nielsen: That's a self-fulfilling argument. People don't filter on contested claims because nobody surfaces them. The whole pedagogical point of this tool is to make the 80 contested ones *findable*. You're optimizing for the median user, which means you're optimizing the median user's blind spot.

Architect → Librarian: A saved-search feature is six weeks of work. We have a 60–80 minute exercise. Can we agree that's out of scope?

Librarian → Architect: Yes — but the *honest hint* under the keyword box is two lines of HTML. I'll trade saved-search for the hint. I want users to know what they're getting, not to discover by failed query that "OR" doesn't work.

Nielsen → Librarian: Conceded. Add the hint. It's free and it prevents the worst-case failure mode (user types Boolean, gets zero results, leaves thinking the corpus is empty).

Architect → Nielsen: Compromise on `signal`: put it in Advanced, not Primary. I'll accept that. But I want a "Show contested only" quick chip next to the keyword box — that's the philosophy student's view as a one-click affordance without adding a dropdown.

Philosophy student → Architect: I'll take the chip. But add one more thing: when the result set is non-empty, show *of these N, M are contested* in the summary line. That keeps disagreement visible even when the user didn't filter for it.

Nielsen → Philosophy student: Acceptable as a future enhancement. Out of scope for this build, but record it as the next iteration.

Resolution

Primary bar (visible by default): keyword (with hint *"substring match, no Boolean operators"* — librarian's contribution), `warrant_class`, `credence ≥`, `warrant_status`. Four filters. Nielsen's rule.

Advanced (collapsed): topic, qualifier, defeat_type, studyType, support ≥, attacks ≤. Philosophy student's filters live here, one click away — not buried, but not pushing the median user out.

Always shown: filtered count + original total + filtered-out count.

Deferred to next iteration: "Show contested only" quick chip; "of these, M contested" annotation; saved searches.

Genuine disagreements that drove design decisions

Disagreement	Tension	Resolution
Nielsen vs Philosophy student	Frequency-of-use (rare → hide) vs pedagogical surfacing (rare → highlight)	Progressive disclosure (Advanced) + future contested-count annotation
Architect vs Nielsen	`signal` is misleading vs liability-critical	`signal` lives in Advanced, not Primary
Architect vs Librarian	Saved-search feature scope	Cut feature; keep the cheap honest-hint
Librarian vs everyone	Boolean operators in keyword box	Hint under the box documents the limitation transparently

---

5. Specification

Component name: `EvidenceSearchFilter`

Purpose: Reduce a list of ~1,900 evidence records to those matching the user's combined filter criteria. Render the filtered list and a running count of filtered-vs-total.

Inputs: - The full `evidence` array (loaded once at page load). - User-controlled filter state (10 fields, listed below). - Optional URL query parameters that pre-populate filter state (for shareable filtered views and reproducible test runs).

Outputs: - A filtered subarray (function output, deterministic, pure). - DOM updates: rendered cards + a summary line `"Showing N of 1,900 (M filtered out)"`.

Data source: `data/ka_payloads/evidence.json` → `.evidence` array, fetched at page load. Falls back to absolute URL `https://xrlab.ucsd.edu/ka/data/ka_payloads/evidence.json`.

Required fields per record: `finding`, `warrant_class`, `credence`, `warrant_status`, `qualifier`, `defeat_type`, `studyType`, `support_count`, `attack_count`, `primary_topic`, `topic_labels`, `citation`.

Core requirements:

#	Requirement
R1	Primary bar exposes exactly four filters: keyword (substring on `finding`), `warrant_class` (select), `credence ≥` (number 0–1), `warrant_status` (select).
R2	Advanced (collapsed by default) exposes: topic substring (matches `primary_topic` OR any `topic_labels` entry, case-insensitive), `qualifier` (select), `defeat_type` (select; "(none)" matches empty string), `studyType` (select), `support_count ≥`, `attack_count ≤`.
R3	All filters AND together. An empty filter does not constrain.
R4	Selects are populated from the actual distinct values in the loaded data — not hardcoded — so unexpected values (e.g., `DEFEATED` status) appear automatically.
R5	Summary always displays both the filtered count and the original total: `"Showing N of 1900"`.
R6	Reset button clears every field and re-renders the full list.
R7	Filter logic is a pure function `applyFilters(records, filterState) → records[]`, exposed on `window.__exD` for tests.
R8	When the filtered set is empty, render an explicit empty state, not just blank space.
R9	URL query parameters matching field IDs (`q`, `warrant_class`, `cred_min`, …) pre-populate filters on load. `adv=1` (or any advanced field present) opens the Advanced section.

UI behavior: - Filters update results on every keystroke / change (debounce not needed — 1,900 records filter in <5 ms in V8). - Result list capped at first 200 cards rendered for layout performance; summary still reports the full filtered count. - Substring match on `finding` and `topic` is case-insensitive.

Success conditions: - Eight acceptance tests pass against the real `evidence.json`. - Manual run: every architect / philosophy-student / librarian query from the panel returns a non-empty, plausible result set (zero-result queries must render the empty state). - Page becomes interactive within ~1 s of load on the real 5.7 MB JSON.

Edge cases: - `defeat_type === ''` (empty string, 1,820 records) — represented in the dropdown as `(none)` so it can be filtered explicitly. - `credence` missing → treat as 0 (filtered out by any positive minimum). - `topic_labels` missing → fall back to `primary_topic` only. - Free-text inputs trimmed; empty input is treated as "no filter," not "match empty." - Boundary on `credence ≥`: comparator is `≥`, not `>`. Records at exactly the threshold are included.

Failure states: - Network failure on JSON load → summary shows a red error message; page remains usable for explanation; selects are empty. - Filter combination yielding zero results → empty-state card with "Try widening credence or removing a filter."

What NOT to do: - Do **not** filter on `construct` (verified all 1,900 records have value `"Unknown"`). - Do **not** hardcode enum option lists from the context file (it disagrees with the real data on at least 4 fields). - Do **not** silently truncate results without telling the user. - Do **not** put `defeat_type` or `qualifier` in the primary bar (panel decision). - Do **not** advertise Boolean operators in the keyword box without supporting them (panel decision — librarian).

---

6. Acceptance Tests

All tests run against the real `evidence.json` (1,900 records). Verification method: invoke `window.__exD.applyFilters(ALL, state)` in the live browser and compare `result.length` to expected. A Python harness in §8 mirrors the JS function exactly as a cross-check.

#	Name	Filter state	Expected	Why it matters
T1	Normal success — mechanism, credence ≥ 0.6	`{warrant_class:'mechanism', cred_min:0.6}`	**161**	Two-field AND; covers Nielsen's "primary" filters working together.
T2	Practitioner — biophilia × stress	`{topic:'biophilia', q:'stress'}`	**40**	Architect's actual user story. Validates topic search routes through `primary_topic` / `topic_labels`, not the broken `construct` field.
T3	High-confidence / strong-data — WARRANTED, 0 attacks, credence ≥ 0.8	`{warrant_status:'WARRANTED', att_max:0, cred_min:0.8}`	**255**	Three-way AND on the "trustworthy core." Confirms numeric comparators work in the right direction.
T4	Contested / defeated — supported_but_contested + UNDERCUTTING	`{qualifier:'supported_but_contested', defeat_type:'UNDERCUTTING'}`	**18**	Philosophy student's user story. Validates Advanced filters work and that the rare cohort is reachable.
T5	Edge — no-result query (analogical, credence ≥ 0.91)	`{warrant_class:'analogical', cred_min:0.91}`	**0**	Empty state must render, not blank screen. There are 4 analogical claims total and none with credence ≥ 0.91.
T6	Reset — no filters returns all	`{}`	**1900**	R3/R6: empty inputs don't constrain.
T7	Boundary — `≥ 0.61` cuts the 7 records at exactly 0.60	`{warrant_class:'mechanism', cred_min:0.61}`	**154**	Forces the implementation to use `≥` not `>`. T1 returns 161 at threshold 0.60 (includes the 7 records at exactly 0.60); T7 returns 154 at 0.61. The 7-record gap *is* the boundary.
T8	Empty-string handling — `defeat_type=(none)`	`{defeat_type:'__none__'}`	**1820**	The 1,820-record default cohort has `defeat_type === ''`. Without the synthetic `(none)` mapping it would be unreachable from the dropdown. This test exercises the only non-trivial dropdown semantic.

7. Implementation

`/Users/dhruvsood/cogs160/week2_exD/filter.html` — single-file static page. Loads `evidence.json` (local first, then absolute URL fallback). All filter logic lives in a pure function `applyFilters(records, f)` exposed on `window.__exD` for the tests. Selects are populated from the actual distinct field values — so when the data contains `DEFEATED` (which the context file said wouldn't exist), it shows up automatically.

Key design decisions traceable to the panel: - **Primary bar**: keyword, warrant_class, credence ≥, warrant_status (Nielsen's "four max" rule). - **Advanced disclosure** (`<details>`): topic, qualifier, defeat_type, studyType, support ≥, attacks ≤ (philosophy student's filters live here, but they're one click

away — they're not buried). - **Always-visible count**: `Showing N of 1,900 (M filtered out)` per Nielsen. - `(none)` **in defeat_type dropdown** for empty-string records — the 1,820-record default cohort would otherwise be unreachable. - **URL state sync** — librarian's "shareable views" idea, now real (R9). Also makes the screenshot harness in §8 deterministic.

The implementation deliberately does **not** filter on `construct` (per the diagnosis below). It uses `primary_topic` and `topic_labels` for topical search instead.

Visual evidence (full screenshots in `screenshots/`)

Query	Screenshot	Live count
Initial load — no filters	screenshots/01_initial.png	Showing 1900 of 1900
Architect — biophilia × stress	screenshots/02_biophilia_stress.png · bar	Showing 40 of 1900
Philosophy — contested + undercutting	screenshots/03_contested_undercut.png · bar	Showing 18 of 1900
High-trust core — WARRANTED, 0 attacks, ≥0.8	screenshots/04_high_trust.png · bar	Showing 255 of 1900
Empty-state — analogical ≥0.91	screenshots/05_empty_state.png · bar	Showing 0 of 1900

Screenshots are deterministic: filter state is encoded in the URL (R9), and the harness in `/tmp/ppt/shoot.js` (Playwright/Chromium) re-renders them from a clean browser context. Live `#summary` text was read via `page.locator('#summary').innerText()` and printed alongside each capture.

Prompts used (including failed attempts)

Prompt 1 — context initialization (used):

> Read `/Users/dhruvsood/Downloads/context_exD_search_filter.md`.

Prompt 2 — task explanation (used, succeeded):

> Explain back what this task is, what the hardest part is, and what assumptions you'd verify before building. Don't write code yet.

Prompt 3 — verify assumptions (this is what caught the bugs):

> Before trusting the context file, run a count against the real `evidence.json` to confirm: `construct` has ~200 unique values, `qualifier` has exactly 3 values, `warrant_status` has exactly 2.

Prompt 4 — failed first attempt at panel (rejected):

> Run a panel for Exercise D and produce a transcript.

> *(Result: a generic "we should have search and filters" exchange with no real disagreement. Rejected and re-prompted.)*

Prompt 5 — re-prompted panel (used):

> The previous panel had no actual disagreement. Re-run with: each panelist must name a filter the others' rankings get *wrong*. Resolve only with an explicit tradeoff.

Prompt 6 — extend the panel:

> The three roles in the assignment all argue from a position. None of them argue from observed user behavior. Add a research librarian voice and have them rebut one of the three. Resolve via a concrete concession.

Prompt 7 — spec / tests / build:

> Now write the spec, then write 4–6 acceptance tests with concrete expected counts grounded in the real data, then build a single-file `filter.html`. Tests must be reproducible — expose the filter function on `window.__exD`. Add boundary tests.

Prompt 8 — verify (live browser, not just Python):

> Run the acceptance tests against the real data using two harnesses: (a) Python that mirrors the JS filter function exactly, (b) the live browser via `window.__exD.applyFilters`. Both must agree on every count.

Prompt 9 — visual verification (failed attempts followed by playwright):

> Capture screenshots at filter states: initial, biophilia/stress, contested/undercutting, high-trust core, empty-state. Use headless Chrome.
>
> *(First attempt:* `chrome --headless --screenshot=...` *produced screenshots that visually showed cards but didn't reflect URL-param filter state — see Diagnosis Failure 4. Resolved by switching to Playwright with* `waitForFunction` + `networkidle` + *locator-based summary read.)*

---

8. Verification Results

Two harnesses, both run against the real `evidence.json` (1,900 records) loaded from `/Users/dhruvsood/Downloads/evidence.json` and served at `http://localhost:8765/evidence.json`.

Harness A — Python (mirrors JS field-by-field)

```
PASS  T1 mechanism + credence ≥ 0.6:                       expected 161,  got 161
PASS  T2 biophilia(topic) + "stress"(q):                    expected 40,   got 40
PASS  T3 WARRANTED + 0 attacks + credence ≥ 0.8:            expected 255,  got 255
PASS  T4 contested + UNDERCUTTING:                          expected 18,   got 18
PASS  T5 analogical + credence ≥ 0.91:                      expected 0,    got 0
PASS  T6 reset:                                             expected 1900, got 1900
PASS  T7 boundary mechanism + credence ≥ 0.61:              expected 154,  got 154
PASS  T8 (none) defeat_type:                                expected 1820, got 1820
```

Harness B — live browser via `window.__exD.applyFilters`

Run via `mcp__Claude_Preview__preview_eval` against the running page at `http://localhost:8765/filter.html`. Verbatim return value (formatted):

```
[
  {"test":"T1 mechanism + credence>=0.6",               "expected":161,  "got":16
  {"test":"T2 biophilia(topic) + stress(q)",            "expected":40,   "got":40
  {"test":"T3 WARRANTED + 0 attacks + cred>=0.8",       "expected":255,  "got":25
  {"test":"T4 contested + UNDERCUTTING",                "expected":18,   "got":18
  {"test":"T5 analogical + cred>=0.91",                 "expected":0,    "got":0,
  {"test":"T6 reset",                                   "expected":1900, "got":19
  {"test":"T7 boundary: mechanism cred>=0.61",          "expected":154,  "got":15
  {"test":"T8 empty defeat_type via (none)",            "expected":1820, "got":18
]
```

Both harnesses agree on every count. Screenshot summary text (read via `page.locator('#summary').innerText()` in Playwright) matched the harness counts for every panel query (40, 18, 255, 0).

---

9. Diagnosis and Iteration

Failure 1 — initial T1 expected count was 154, actual 161

What happened: During spec-writing, I ran an exploratory probe with `credence > 0.6` (strict) and got 154. I wrote that into the test as the expected value. When the test ran against the implementation — which uses `credence ≥ 0.6` per the spec (R2 says `cred_min` field is "credence ≥") — it returned 161. The 7 records with `credence === 0.60` exactly were the difference.

Diagnosis category: *Test was wrong*. The spec was correct (`≥`), the implementation was correct (`≥`), the data was correct. The expected value in the test was computed under the wrong comparator.

Correction: Updated T1's expected to 161. Did not change the spec or the code. Lesson recorded: when generating expected counts, always compute them with the *exact same operator* the spec mandates, not whatever feels natural at the keyboard. Added **T7** as a new explicit boundary test that *only passes if the comparator is exactly* `≥`: `cred_min:0.61` returns 154 (cuts the 7 boundary records); `cred_min:0.60` returns 161 (includes them). The gap *is* the test.

Failure 2 — `construct` field is unusable

What happened: The context file claimed `construct` had ~200 unique values like "Lighting, Biophilia, Acoustics." A naïve implementation would have put `construct` as a primary filter dropdown. Probing `Counter(e['construct'] for e in ev)` returned `{'Unknown': 1900}`.

Diagnosis category: *Data assumption was wrong*. The context file was outdated relative to the current `evidence.json` payload.

Correction: Removed `construct` from the spec entirely. Added a topic substring filter on `primary_topic` + `topic_labels`. Verified the architect's biophilia query (T2) now returns 40 results that actually mention biophilia.

Failure 3 — context file enums incomplete

`qualifier`, `warrant_status`, `studyType`, and `signal` all have more values than the context file documented (e.g., `warrant_status: DEFEATED` exists but wasn't listed).

Diagnosis category: *Data assumption was wrong*.

Correction: R4 in the spec now mandates that selects be populated from actual distinct values in the loaded data, not hardcoded. This makes the component robust to future schema additions without a code change.

Failure 4 — headless screenshots didn't reflect URL-param filter state

What happened: When I added URL state (R9) to support reproducible screenshots, my first capture script used `Google Chrome --headless --screenshot=...`. The full-page PNGs *appeared* to show filtered card lists (cards looked biophilia-related), but a closer read of the summary line still showed "1900 of 1900." Live `eval` against the same page in `mcp__Claude_Preview__preview_eval` returned the *correct* count ("Showing 40 of 1900").

Diagnosis category: *Tooling was wrong*. Chrome's `--virtual-time-budget` advances virtual clock time, but the order in which `fetch → JSON.parse → populateSelects → applyUrlState → update` resolves under virtual time isn't deterministic relative to the screenshot capture. The capture was firing after some renders but before the filter-driven re-render landed.

Correction: Replaced `chrome --headless --screenshot` with a Playwright/Chromium harness (`/tmp/ppt/shoot.js`) that: 1. `await page.goto(url, { waitUntil: 'networkidle' })` 2. `await page.waitForFunction(() => window.__exD?.ALL?.length > 0)` 3. Reads `page.locator('#summary').innerText()` and prints it next to each screenshot path.

The printed summary line is the visible-evidence proof that the screenshot reflects the filtered state. All five summaries match the test counts: 1900, 40, 18, 255, 0.

What I'd iterate next (out of scope for this submission)

- Philosophy student's "of these N, M are contested" annotation in the summary line. Panel agreed; deferred.
- Architect's "Show contested only" quick chip. Same.
- Librarian's "saved searches" — explicit out-of-scope per round 2 concession. URL state (R9) covers half the use case (shareable bookmarks).
- Result sort order — currently insertion order; should default to credence-descending.

---

10. Reflection Paragraph 1 — Panel Surprise

The surprise wasn't *that* the panel disagreed — it's that the disagreement reframed the assignment for me. I'd read the prompt as "build a filter bar, decide which filters to show," which is a usability question. Nielsen and the architect treated it that way. The philosophy student didn't. She argued that because contested claims are 80 records out of 1,900, hiding them behind "Advanced" *teaches* users that disagreement is statistically rare and therefore unimportant — which inverts the entire pedagogical point of K-ATLAS. That moved me from "what's most-clicked goes on top" to "what's most-clicked goes on top, *but* the rare epistemically-important stuff needs an affordance, not a hiding place." When I added the librarian's voice in round two, a second surprise landed: every filter discussion had been about *which fields to show*, but the librarian made it about *what query patterns users will attempt and fail at*. The Boolean-operator critique was something none of the role-based panelists would have produced — a usability expert thinks about clicks, a domain expert thinks about their own queries, but the librarian thinks about queries that fail silently. I'd have shipped without the substring-match hint without that voice. The panel didn't just produce a feature list; it produced a hierarchy where each panelist's worst blind spot was visible to a different panelist.

---

11. Reflection Paragraph 2 — Specific Failure and Diagnosis

The failure I learned the most from was the smallest one: T1 expected 154, got 161. I had written the expected count by running an exploratory `>` probe in Python, then written the spec with `≥`, and never reconciled the two. The implementation was correct; the test was lying. I caught it only because I ran the verification before writing the doc — if I'd skipped that step or just trusted the spec section I'd already written, I'd have shipped a "passing" submission with a quietly wrong test. Diagnosing it was three lines of work: print the records with `credence === 0.60` exactly, count them (7), confirm `154 + 7 = 161`. The fix was one number in the doc. But the lesson is bigger than the bug — it's that *probes* and *tests* are not the same artifact and shouldn't share their expected values by copy-paste. A probe answers "what's there"; a test asserts "what the spec says should be there." When those two come apart, the test must be regenerated against the spec, not against my memory of the probe. I now treat any handwritten "expected" number as suspect until it's been re-derived from the spec's exact comparator. As a defensive move, I added T7 as a deliberate boundary test: it only passes if the implementation uses `≥` and not `>` — the 7-record gap between T1 (161) and T7 (154) is the test. If a future refactor accidentally swaps the operator, T7 fails before any user notices.

---

12. Final Checklist

Assignment requirement	Where in this doc	Status
Context file pasted as the first AI message	§2 Context Initialization	✅
Panel transcript with real disagreement	§4 Panel Transcript (4 voices, 2 rounds, 4 distinct disagreements with concrete tradeoffs)	✅
Specification with inputs, outputs, edge cases, success conditions	§5 Specification (9 numbered requirements R1–R9)	✅
At least 4 acceptance tests with expected outcomes	§6 — 8 tests including normal, high-confidence, contested/defeated, edge/no-result, reset, boundary, empty-string	✅ (2× minimum)

Assignment requirement	Where in this doc	Status
All AI prompts included, including failed attempts	§7 (Prompts 1–9; Prompt 4 + Prompt 9's first attempt explicitly marked failed)	✅
Verification results: pass/fail with evidence	§8 — Python harness + live browser harness, both 8/8 PASS, plus screenshot summary lines matching counts	✅
Reflection paragraph 1 — surprise from the panel	§10	✅
Reflection paragraph 2 — specific failure and diagnosis	§11	✅
Implementation file	`filter.html`	✅
Visual evidence	`screenshots/` (5 full-page + 4 filter-bar crops)	✅